

Multi-GPU Training Paradigms

When training large-scale AI models (like LLMs), a single GPU often runs out of memory or takes too long to process. To solve this, we use multiple GPUs. The methods by which these GPUs communicate and divide the workload are called **Training Paradigms**.

These paradigms are generally divided into two groups:

1. **Dividing the Data:** Keeping the model the same but splitting the training data.
2. **Dividing the Model:** Splitting the model itself because it's too large for one GPU.

1. Model Parallelism (Inter-op Parallelism)

This is the most intuitive way to split a model. Imagine a neural network with 10 layers.

- **How it works:** You place layers 1–5 on **GPU-1** and layers 6–10 on **GPU-2**.
- **The Process:** Data is processed by GPU-1 first, and the output is then sent to GPU-2 for the remaining steps.
- **Pros:** Allows you to train models that are physically too large to fit into the VRAM of a single GPU.
- **Cons:** It creates a **bottleneck**. Because the layers are sequential, GPU-2 must wait for GPU-1 to finish. This "waiting time" means the GPUs aren't working at 100% capacity simultaneously.

2. Data Parallelism

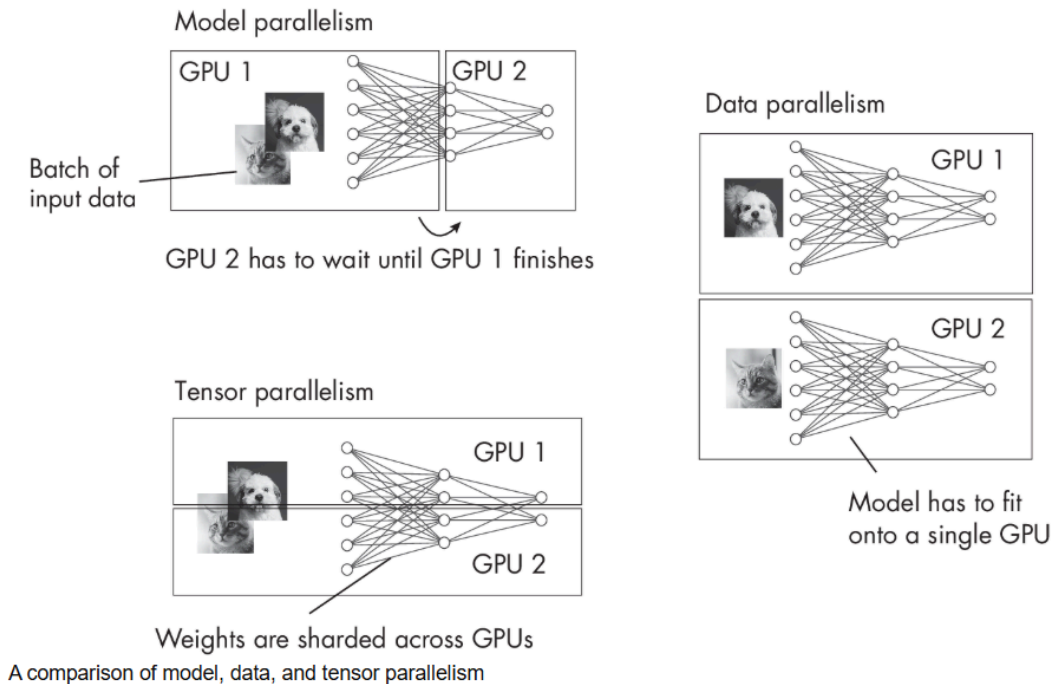
This has been the industry standard for years when the model fits on a single GPU but the dataset is massive.

- **How it works:** Every GPU gets an **identical copy** of the entire model. The large batch of data (minibatch) is split into smaller "microbatches."
- **The Process:** Each GPU processes its own chunk of data and calculates the "loss gradients." Afterward, all GPUs communicate to average their results and update the model weights together.
- **Pros:** Highly efficient because all GPUs work at the same time (parallel processing).
- **Cons:** Every single GPU must have enough memory to hold the entire model. If the model is too big, this method fails.

3. Tensor Parallelism (Intra-op Parallelism)

This is a more surgical version of model parallelism. Instead of splitting the model by layers, we split the **mathematical operations (tensors)** within a layer.

- **How it works:** If a layer requires a massive matrix multiplication, that matrix is sliced into pieces. Half the matrix goes to GPU-1 and the other half to GPU-2 (as shown in **Figure**).
- **Pros:** Solves memory issues for massive models while allowing GPUs to work on parts of the same operation simultaneously (unlike the "waiting" in model parallelism).
- **Cons:** Requires very high-speed communication between GPUs. If the connection (interconnect) is slow, the training slows down significantly.



4. Pipeline Parallelism

This is a "smarter" hybrid of model and data parallelism designed to fix the "waiting" issue in standard model parallelism.

- **How it works:** It uses a "factory assembly line" approach. While GPU-2 is working on the first piece of data it received from GPU-1, GPU-1 doesn't sit idle—it immediately starts processing the *next* piece of data.
- **Pros:** Much more efficient than simple model parallelism.
- **Cons:** Complex to implement and can still have small gaps of idle time (called "bubbles").

5. Sequence Parallelism

This is specifically designed for Transformers dealing with very long sequences (like long books or long code files).

- **How it works:** It splits the long input sequence (the sentences) into smaller chunks and distributes them across GPUs (**Figure 7.3**).
- **Pros:** Reduces the massive memory requirement that comes with the "Self-Attention" mechanism in Transformers when handling long text.
- **Cons:** Breaking sequences can sometimes slightly reduce the model's accuracy or "understanding" of the full context.

Paradigm	What is divided?	Primary Goal
Data Parallelism	The Data (Batches)	Speed up training on large datasets.
Model Parallelism	The Model (Layers)	Fit a large model across multiple GPUs.
Tensor Parallelism	The Math (Matrices)	Efficiently run massive models in parallel.

Practical Recommendations

1. **For Small Models:** If your model fits on one GPU, always use **Data Parallelism**. It is the fastest and easiest.
2. **For Large Models (LLMs):** If the model doesn't fit on one GPU, use **Tensor Parallelism**. It is generally more efficient than basic model parallelism.
3. **Modern Approach:** Most state-of-the-art systems (like DeepSpeed) use a **Hybrid Blend** of Data Parallelism and Tensor Parallelism to get the best of both worlds—handling huge models while keeping the speed high.